ARCHITECTURAL AUDIT & DATA VERIFICATION

HomeFaciliti API & Schema Analysis

A Comprehensive Comparison of Data Models Between the Admin Panel Backend and the Main App Backend

Prepared For:	HomeFaciliti Development Team
Prepared By:	Antigravity AI Coding Assistant
Date:	May 31, 2026
Status:	**OUT OF SYNC DETECTED**

Notice: This audit details critical discrepancies between the raw text file APIs in *api_list.txt* and the production MongoDB schemas in the *homefaciliti-backend* repository. Alignment is highly recommended to avoid run-time database validation failures.

1. Executive Summary

An architectural comparison has been performed between the **Admin Panel Backend** API specification (compiled in api_list.txt) and the **Main App Backend** implementation schemas (found in homefaciliti-backend/models).

Core Finding: The two systems are completely **out of sync**. The Admin Panel Backend uses flat string representation, legacy field names (e.g. mobile instead of phone), and custom KYC columns embedded directly in collections. The Main App Backend enforces strict MongoDB Object relations (e.g. userId, vendorId, serviceId) and modern authentication schemas (OTP-based). Running both systems against a shared database in their current states will result in schema corruption or query errors.

Key Discrepancies Overview Table

Entity	Admin API (api_list.txt)	Main Backend Schema (Mongoose)	Compatibility Status
User	Fields: name, email, mobile, address. CRUD-based.	Fields: name, phone, password, role, city. OTP-based authentication.	**Incompatible** Mobile vs Phone
Vendor / Partner	Contains bank details, KYC status, experience, ratings, direct metrics and email/mobile embedded.	Serves as a relational link: userId (ref User), services (ref Service), rating, earnings, isApproved.	**Incompatible** Admin embeds, main references
Booking / Order	Fields: serviceRequestNumber, serviceName, serviceAmount, slotTime, serviceDate, vendorName.	Fields: userId, vendorId, serviceId, status, totalAmount, commissionAmount.	**Incompatible** Strings vs ObjectId references
Service	Fields: title, price, image, description, status.	Fields: title, category, price, duration, city.	**Partial Sync** Missing description/image
Review	Fields: userName, partnerName, serviceName, rating, reviewText, status.	Fields: userId, vendorId, rating, comment.	**Incompatible** Text vs Object references

2. Deep Dive Discrepancies & Payload Incompatibilities

Here is the granular, field-level analysis demonstrating that completely different data payloads are currently going through the respective APIs:

User Payload Conflict

• **Admin API:** Expects flat string object for user creation:
{ "name": "Hira", "email": "hira@gmail.com", "mobile": "9199953391", "address": "Jaipur" }
• **Main App Backend:** Expects a secure hashed document with role, and links registration to phone numbers verified by SMS OTP:
{ "name": "...", "phone": "...", "password": "...", "role": "user" }
• **Key Risk:** Admin panel POST/PUT requests will fail schema validation as they lack the password field, and provide mobile instead of phone.

Booking / Order Payload Conflict (Highly Critical)

• **Admin API:** Expects fully denormalized string data, allowing instant display of names, dates, and slots:
{ "serviceRequestNumber": "SR123", "serviceName": "Leakage Repair", "serviceAmount": 299.0, "slotTime": "2 PM - 3 PM", "vendorName": "Rajesh" }
• **Main App Backend:** Relies on a highly normalized relational structure using MongoDB ObjectIds to preserve data integrity and prevent duplicated strings:
{ "userId": ObjectId("..."), "vendorId": ObjectId("..."), "serviceId": ObjectId("..."), "totalAmount": 299 }
• **Key Risk:** An order written by the main application cannot be loaded or edited by the admin panel because the admin panel does not populate model relations (will throw reference loading errors). Conversely, orders written by the admin panel will violate mongoose strict schema rules of the main backend.

Partner / Vendor Payload Conflict

• **Admin API:** Embeds name, email, phone, city, bank account details, pan card, total ratings, reviews counts, experience, etc. in a single monolithic document.
• **Main App Backend:** Implements the Single Responsibility Principle. Vendor details are separated into the User collection, while the Vendor collection only keeps track of referencing pointers and rating numbers.
• **Key Risk:** Approving or managing partners via the Admin Panel will fail to populate the primary User collection on the main backend. The vendor will not be able to log in as they lack a credential record in the main user database.

3. Recommendations & Sync Strategy

To run the Admin Panel and the Main App concurrently on the same database successfully without data corruption, the following migration steps are urgently required:

1. Implement DB Normalization in Admin Panel:

Instead of storing raw string fields for users, partners, and services inside bookings and reviews, the admin backend must be upgraded to support MongoDB Object References. For instance, booking creation should request `userId`, `vendorId`, and `serviceId` rather than plain strings.

2. Unify Field Naming:

Align variable schemas. Standardize on phone instead of mobile, and comment instead of reviewText across all backend servers. This prevents field duplication in databases.

3. Adopt Shared Model Definitions:

Extract the database Mongoose models into a shared NPM module (e.g., @homefaciliti/core-models). This ensures that any change in schema by the mobile app or main backend is automatically compiled and validated by the admin panel, avoiding silod out-of-sync discrepancies.

Audit Verdict & Approval

This report confirms that **different data structures** are going through the APIs. The systems are currently architecturally incompatible and must be synchronized.

Lead Auditor:	Antigravity AI Coding Assistant
Status:	**ACTION REQUIRED**
PDF Report Version:	1.0.0